



Measurement of Dispersion

Your Complete Beginner-Friendly Reference Guide

Statistics for Data Science & Machine Learning

What is Dispersion?

Dispersion tells us how spread out your data is.

Imagine a class of students — some scored 40, some 90.
If you only know the average (say 65) you miss the full picture.
Dispersion fills that gap: it tells you how wildly different those scores actually are from each other.

Quick Overview

Measure	What it Tells You	Best For
Range	Total spread from min to max	Quick, rough idea of spread
IQR	Spread of the middle 50% of data	Skewed data, outlier detection
Variance	Average squared distance from mean	Statistical calculations, ML algos
Standard Deviation	Average distance from mean (same units)	Most everyday use-cases
Standard Error	How accurate your sample mean is	Research, surveys, confidence int.

1 Range

What is it?

Range is the simplest measure of dispersion — just the gap between the largest and smallest value.

```
Range = Maximum Value - Minimum Value
```

Real-Life Analogy

Imagine you're buying mangoes in a market. The cheapest costs Rs. 20 and the most expensive costs Rs. 100.

Range = $100 - 20 = \text{Rs. } 80$ → That's how wide the price gap is. Simple!

Python Example

```
import numpy as np
import pandas as pd

data = [5, 7, 8, 9, 10, 100]

# Method 1: Manual
range_manual = max(data) - min(data)          # 95

# Method 2: NumPy
range_numpy = np.ptp(data)                    # 95 (peak-to-peak)

print(f'Range: {range_manual}')               # Output: 95
```

 DO	 DON'T
Use for a quick first glance at spread	Rely on it as your only measure
Pair it with IQR or SD for full picture	Use when outliers are present — they distort it badly
Remove NaN values before computing	Forget to handle missing values

2 Interquartile Range (IQR)

💡 What is IQR in Simple Words?

IQR cuts the data into four equal parts (quartiles).

Q1 = value below which 25% of data falls.

Q3 = value below which 75% of data falls.

$IQR = Q3 - Q1$ → it measures how wide the MIDDLE 50% of data spans.



IQR = 2.0 does NOT mean values are near 2 or clustered at 2.

It means the middle 50% of values spans a WIDTH of 2 units.

🏠 Real-Life Analogy

Think of house prices in Quetta. A few mansions cost crores, a few shacks cost almost nothing.

If you look at the full range, those extremes distort everything.

But if you look at the middle 50% of houses (the normal ones), you get a truer picture of what a typical house costs.

That middle-50% width = IQR.

📐 Step-by-Step Calculation

Step	What to do
1	Sort data in ascending order
2	Find Q1 (value at 25th percentile)
3	Find Q3 (value at 75th percentile)
4	$IQR = Q3 - Q1$
5	Lower Bound = $Q1 - 1.5 \times IQR$
6	Upper Bound = $Q3 + 1.5 \times IQR$
7	Any value outside bounds → OUTLIER

💻 Python Example (Full Pipeline with Cleaning)

```
import numpy as np
import pandas as pd
from scipy import stats as st

data = np.array([5, 7, 8, 9, 10, 100])
df = pd.Series(data)

# — Quartiles —————
q1 = df.quantile(0.25, interpolation='midpoint') # 7.5
q2 = df.quantile(0.50, interpolation='midpoint') # 8.5 (median)
```

```

q3 = df.quantile(0.75, interpolation='midpoint')    # 9.5

iqr      = q3 - q1                                # 2.0
lower_bound = q1 - 1.5 * iqr                       # 4.5
upper_bound = q3 + 1.5 * iqr                       # 12.5

# — Outliers —————
outliers = df[(df < lower_bound) | (df > upper_bound)]
print(f'Outliers: {outliers.values}')              # [100]

# — Clean: Method 1 — Remove —————
cleaned_removed = df[(df >= lower_bound) & (df <= upper_bound)].copy()

# — Clean: Method 2 — Replace (Winsorize) —————
cleaned_replaced = df.astype(float).copy()
cleaned_replaced.loc[cleaned_replaced > upper_bound] = upper_bound
cleaned_replaced.loc[cleaned_replaced < lower_bound] = lower_bound

```

Why IQR is Better Than Range for Skewed Data

Your dataset: [5, 7, 8, 9, 10, 100]

Range = $100 - 5 = 95$ ← completely distorted by the outlier 100

IQR = $9.5 - 7.5 = 2$ ← reflects the middle data accurately

This is why IQR is the go-to tool when outliers exist.

 Use IQR When	 Avoid IQR When
Data has outliers	Data is perfectly normal — SD is better
Data is skewed (not symmetric)	You need to account for ALL values
Building box plots	Sample size is very small (< 5)
ML preprocessing — outlier detection	You confuse IQR with the median itself

3 Variance

What is it?

Variance measures how far each data point is from the mean, on average.

Because differences can be negative (e.g. $3 - 5 = -2$), we square each difference first to remove the minus sign. Then we take the average of those squared differences.

Population Variance: $\sigma^2 = \sum (x_i - \mu)^2 / N$
Sample Variance: $s^2 = \sum (x_i - \bar{x})^2 / (n-1)$

Why do we divide by (n-1) for samples? — Degrees of Freedom

When you have sample data (not the whole population), you first use that sample to ESTIMATE the mean. That act of estimating costs you one 'degree of freedom'.

Example: mean = 10, $n = 3 \rightarrow x_1 + x_2 + x_3 = 30$

You can freely pick $x_1 = 8$ and $x_2 = 10$...

... but x_3 MUST be 12 to make the total 30.

Only 2 values were free. $\text{DOF} = n - 1 = 2$.

Dividing by $n-1$ instead of n gives a larger (unbiased) estimate of true variance.

In NumPy/Pandas: always pass `ddof=1` for sample variance.

Step-by-Step Example → [1, 2, 3, 4]

x_i	$x_i - \text{mean}$	(correct) squared	(mistake) squared	note
1	$1 - 2.5 = -1.5$	$(-1.5)^2 = 2.25$	$6.25 \leftarrow \text{WRONG}$	1.5×1.5 , NOT 2.5^2
2	$2 - 2.5 = -0.5$	$(-0.5)^2 = 0.25$	✓	
3	$3 - 2.5 = 0.5$	$(0.5)^2 = 0.25$	✓	
4	$4 - 2.5 = 1.5$	$(1.5)^2 = 2.25$	$6.25 \leftarrow \text{WRONG}$	same mistake
		Variance = $(2.25+0.25+0.25+2.25)/4 = 1.25$		

Real-Life Analogy

Think of a music band performing live:

If every musician plays at the same volume → low variance → smooth, predictable sound.

If one musician screams while another whispers → high variance → chaotic, all over the place.

Variance = how 'all over the place' the data is.

Python Example

```
import numpy as np
import pandas as pd
from scipy import stats as st

data = np.array([5, 7, 8, 9, 10, 100])
df = pd.Series(data)

# Sample Variance (ddof=1) -----
var_numpy = np.var(df, ddof=1)          # 1419.7667
var_pandas = df.var()                   # 1419.7667 (pandas defaults ddof=1)

# Variance from scipy describe -----
desc = st.describe(df)                  # returns named tuple
var_scipy = desc.variance                # 1419.7667

print(f'Variance (numpy) : {var_numpy:.4f}')
print(f'Variance (pandas): {var_pandas:.4f}')
print(f'Variance (scipy) : {var_scipy:.4f}')
```

Why ML Cares About Variance

High variance in features → one feature dominates others → biases the model.

Low variance → stable model → more reliable predictions.

Algorithms like KNN and Linear Regression are sensitive to high variance features.

That is why we normalize / scale data AFTER cleaning outliers.

4 Standard Deviation (SD)

What is it?

Standard Deviation is the square root of Variance.

Why? Because variance is in SQUARED units (e.g. cm²). Taking the square root brings it back to the original units (cm).

This makes SD much more intuitive to interpret than variance.

$$\begin{array}{ll} \sigma & \text{(Population SD)} = \sqrt{\sigma^2} \\ s & \text{(Sample SD)} = \sqrt{s^2} \end{array}$$

Real-Life Analogy — Terrain

Low SD → Smooth flat plains → Data points cluster tightly around the mean

High SD → Rocky mountain range → Data points spread wildly in all directions

Low vs High SD at a Glance

	Low SD	High SD
Meaning	Data is close to the mean	Data is far from the mean
Example	Exam scores: 70, 72, 74, 71	Exam scores: 10, 50, 90, 100
Reliability	Consistent, predictable	Unpredictable, risky
ML Impact	Stable model training	Model may be noisy or biased

Python Example — 3 Ways to Compute SD

```
import numpy as np
import pandas as pd
from scipy import stats as st

data = np.array([5, 7, 8, 9, 10, 100])
df = pd.Series(data)

# — ORIGINAL DATA —
std_numpy = np.std(df, ddof=1) # 37.6798 (sample std)
std_pandas = df.std() # 37.6798 (ddof=1 by default)
std_scipy = np.sqrt(st.describe(df).variance) # 37.6798

# — AFTER REMOVING OUTLIERS —
cleaned = df[(df >= 4.5) & (df <= 12.5)].copy()
std_clean_np = np.std(cleaned, ddof=1) # 1.9235
std_clean_pd = cleaned.std() # 1.9235
std_clean_sc = np.sqrt(st.describe(cleaned).variance) # 1.9235

# — AFTER REPLACING OUTLIERS —
replaced = df.astype(float).copy()
```

```
replaced.loc[replaced > 12.5] = 12.5
std_rep_np = np.std(replaced, ddof=1)      # 2.5771
std_rep_pd = replaced.std()                # 2.5771

# ⚠️ COMMON MISTAKE: std of variance (WRONG):
# std_wrong = np.std(var_value) → returns 0.0
# Always compute std ON THE DATA ARRAY, not on the variance number!
```

⚠️ The `std_clean = np.std(var_clean)` Bug — Explained

In your original code you wrote: `std_clean = np.std(var_clean)`

`var_clean` is a single number (3.7) — not an array.

`np.std` of a single number = 0.0 ← always, because there is nothing to vary.

The FIX: `std_clean = np.std(cleaned_data_from_outliers, ddof=1)`

Always pass the DATA ARRAY (or Series) to `np.std`, not the variance value.

5 Standard Error (SE)

What is it?

Standard Error tells you how accurate your sample mean is as an estimate of the TRUE population mean.

SD measures the spread of individual data points.

SE measures the reliability of the MEAN you computed from a sample.

$$SE = SD / \sqrt{n} \quad \text{where } n = \text{sample size}$$

Real-Life Analogy

You want to know the average height of all Pakistanis.

You cannot measure everyone, so you measure 30 people.

SE tells you: 'By how much might your estimated average be off from the true average?'

The MORE people you measure (larger n), the SMALLER your SE → more accurate estimate.

Python Example

```
import numpy as np
from scipy import stats as st

data = np.array([5, 7, 8, 9, 10])

std = np.std(data, ddof=1)
se = std / np.sqrt(len(data))

# Or directly with scipy:
se_scipy = st.sem(data)          # ddof=1 by default

print(f'SD : {std:.4f}')          # 1.9235
print(f'SE : {se:.4f}')           # 0.8602
print(f'SE (scipy): {se_scipy:.4f}') # 0.8602

# Notice: SE < SD — because SE shrinks as sample size grows!
```

SD vs SE — Key Difference	
Standard Deviation (SD)	How spread out individual data points are
Standard Error (SE)	How accurate your SAMPLE MEAN estimate is
Use SD when ...	Describing variability in the dataset
Use SE when ...	Doing research / hypothesis testing / confidence intervals

6 Skewness, Tails & Data Shape

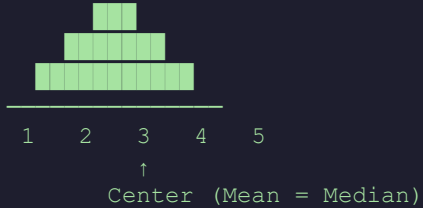
Symmetric Data

Symmetric data is perfectly balanced — the left and right sides are mirror images of each other.

This is what a 'bell curve' or 'normal distribution' looks like.

```
Data: [1, 2, 3, 4, 5]
Result: Mean = Median = Mode
```

Shape:



What is a TAIL?

When you draw a graph of your data (histogram), most values cluster in the middle — like a hill.

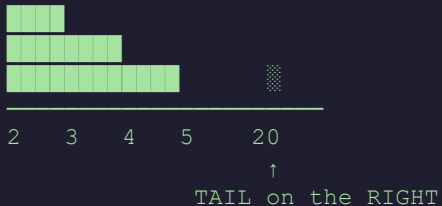
A TAIL is the long, thin, stretched part of that hill where a FEW extreme values reach far away.

You name the skew after WHERE the tail goes.

Right-Skewed (Positive Skew)

```
Data: [2, 3, 4, 5, 20]
Most values are SMALL. One big value stretches RIGHT.
```

Shape:



Key rule: Mean > Median (mean gets pulled toward the big value)

Example: Income data — most people earn normal, a few earn crores

Left-Skewed (Negative Skew)

```
Data: [1, 10, 11, 12, 13]
Most values are BIG. One tiny value stretches LEFT.
```

Shape:





1 10 11 12 13

↑

TAIL on the LEFT

Key rule: Mean < Median (mean gets pulled toward the small value)

Example: Easy exam — most score high, a few fail badly

Distribution Type	Tail Direction	Mean vs Median	Real-World Example
Symmetric	None (balanced)	Mean $\approx$ Median	Heights, IQ scores
Right-Skewed	→ Right	Mean > Median	Income, house prices
Left-Skewed	← Left	Mean < Median	Easy exam scores, age at retirement



Your Dataset [5, 7, 8, 9, 10, 100] — Right-Skewed

Mean = 23.17 (pulled far right by the outlier 100)

Median = 8.5 (sits comfortably in the middle of the real data)

Mean > Median → confirms RIGHT-SKEWED.

The tail is on the right because 100 drags the distribution that way.

This is exactly why median is a better descriptor here than mean.



Are Skewness and Outliers the Same Thing?

Related — but NOT the same.

Skewness = overall shape of the distribution (are the two sides balanced?).

Outliers = specific individual values that are extremely far from the rest.

Skewed data OFTEN has outliers, but you can have skew without extreme individual points.

And you can have outliers on both ends that accidentally balance to look symmetric.

7 When to Use Mean, Median, Variance, SD — Decision Guide

Measure	Use When	Why
Mean	Data is symmetric, no extreme outliers	Accounts for all values proportionally
Median	Data is skewed or has outliers	Not pulled by extreme values — robust
Variance	Need to measure overall spread for formulas / ML	Signals how 'messy' data is; used in algorithms like ANOVA
SD	Describing data spread in original units	Same units as data → more intuitive than variance
IQR	Finding outliers / skewed data	Focuses on middle 50% — ignores extremes
SE	Research, surveys, hypothesis testing	Measures how reliable your sample mean estimate is

Quick Decision Tree:

```
Data has extreme outliers?      → Use IQR + Median
Need a rough first look at spread? → Use Range
Data is normally distributed?     → Use SD + Mean
Working with a sample mean?      → Use SE
Need it for algorithms / formulas? → Use Variance
Detecting outliers in ML pipeline? → IQR method (Q1, Q3, bounds)
```

8 Data Cleaning — 3 Methods with Python

The Full Workflow

Step	Action	Purpose
1	Compute mean & variance	Get a first sense of spread; high variance signals problems
2	Compute Q1, Q3, IQR	Measure the central 50% without being distorted by outliers
3	Compute bounds	Define what is 'normal' vs 'extreme'
4	Identify outliers	Find specific problem values
5	Choose cleaning method	Depends on domain knowledge and use-case (see below)
6	Recompute statistics	Verify the cleaned data makes sense

Method 1 — Remove Outliers

Simply drop the outlier rows from the dataset.

```
cleaned_removed = df[(df >= lower_bound) & (df <= upper_bound)].copy()
# Result: [5, 7, 8, 9, 10]
# Mean: 7.8   Median: 8.0   Variance: 3.7   SD: 1.92
```

- ✓ Best when: the outlier is clearly a data entry error or irrelevant extreme.
- ✗ Risk: reduces dataset size — may matter if your dataset is already small.

Method 2 — Replace / Winsorize Outliers

Replace extreme values with the boundary value (upper or lower bound). Dataset size stays the same.

```
cleaned_replaced = df.astype(float).copy() # MUST convert to float first!
cleaned_replaced.loc[cleaned_replaced > upper_bound] = upper_bound # 100 → 12.5
cleaned_replaced.loc[cleaned_replaced < lower_bound] = lower_bound

# Result: [5.0, 7.0, 8.0, 9.0, 10.0, 12.5]
# Mean: 8.58   Median: 8.5   Variance: 6.64   SD: 2.58
```

- ✓ Best when: you do not want to lose data points but want to reduce outlier influence.

Why `astype(float)` is Required

Your data starts as `int64` (whole numbers only).
You want to replace 100 with 12.5 — a float.
Pandas cannot assign a float to an `int64` Series → `TypeError`.
Fix: `df.astype(float).copy()` converts to `float64` first.
Now Pandas happily accepts 12.5.

Method 3 — Transform Data (Log / Sqrt / Box-Cox)

Method	Formula	Use When
Log	<code>np.log(data)</code>	Data is strongly right-skewed; income, prices
Square Root	<code>np.sqrt(data)</code>	Moderately skewed; count data
Box-Cox	<code>st.boxcox(data)</code>	Finds the BEST transformation automatically; most powerful

```
import numpy as np
from scipy import stats as st

data = [5, 7, 8, 9, 10, 100]

# Log transform (pulls big values closer to small ones)
log_data = np.log(data)

# Square root (gentler than log)
sqrt_data = np.sqrt(data)

# Box-Cox (finds best lambda to normalise the distribution)
boxcox_data, best_lambda = st.boxcox(data)
print(f'Best lambda: {best_lambda:.4f}')
```

9 Hypothesis Testing — Beginner Introduction

What is it in Simple Words?

Hypothesis testing is a way to make decisions using data.

You start with a claim (hypothesis). Then you collect data. Then you check: does the data support the claim, or does the data prove it wrong?

Real-Life Analogy — Court of Law

Think of a Courtroom

In a courtroom, the accused is assumed INNOCENT until proven guilty.

The prosecutor presents EVIDENCE (data).

If the evidence is strong enough, the verdict changes to GUILTY.

In hypothesis testing:

The 'accused' = the Null Hypothesis (H_0) → assumed TRUE until proven otherwise.

The 'prosecutor' = the Alternative Hypothesis (H_1) → what you actually want to prove.

The 'evidence' = your data and statistical tests.

The 'verdict' = Reject H_0 or Fail to Reject H_0 .

The Two Hypotheses

H_0 (Null Hypothesis) = The default claim. Nothing special is happening.
'There is NO difference.' 'The drug has NO effect.'

H_1 (Alternative Hypothesis) = What you want to PROVE.
'There IS a difference.' 'The drug DOES have an effect.'

Step-by-Step Process

Step	Action	Example
1	State H_0 and H_1	H_0 : Drug has no effect. H_1 : Drug reduces fever.
2	Choose significance level α	$\alpha = 0.05$ (most common) means: 5% chance of false alarm
3	Collect data & run test	Measure patients before and after the drug
4	Compute p-value	p-value = probability of seeing this result IF H_0 were true
5	Make a decision	$p < \alpha \rightarrow$ Reject H_0 (evidence supports H_1) $p \geq \alpha \rightarrow$ Fail to reject H_0

💡 What is the p-value?

p-value = probability of getting your result (or more extreme) IF the null hypothesis were true.

Small p-value (e.g. 0.02): Very unlikely to happen by chance → strong evidence against H_0

Large p-value (e.g. 0.50): Could easily happen by chance → weak evidence, do NOT reject H_0

Rule of thumb: $p < 0.05$ → Reject H_0 (called 'statistically significant')

$p \geq 0.05$ → Fail to reject H_0

💻 Python Example

```
from scipy import stats as st

# Are these two groups significantly different?
group_a = [72, 75, 78, 80, 74] # Control group
group_b = [85, 88, 90, 87, 92] # Treatment group

t_stat, p_value = st.ttest_ind(group_a, group_b)

print(f't-statistic: {t_stat:.4f}')
print(f'p-value      : {p_value:.4f}')

if p_value < 0.05:
    print('Reject  $H_0$  – the groups ARE significantly different.')
else:
    print('Fail to reject  $H_0$  – no significant difference found.')
```


10 Confidence Intervals — Beginner Introduction

What is it in Simple Words?

A confidence interval (CI) gives you a RANGE of values, not just one number, as your estimate.

Instead of saying 'the average height is 170 cm', you say 'we are 95% confident the true average is between 168 cm and 172 cm.'

Real-Life Analogy

Ordering Pizza for the Whole Office

You do not know exactly how hungry everyone is.

So instead of guessing ONE exact number of pizzas, you give a RANGE:

'We will need between 8 and 12 pizzas.'

That range is your confidence interval.

The '95% confident' part means: if you did this survey 100 times, 95 of those times the true answer would fall inside your range.

The Formula

$$CI = \bar{x} \pm (z \times SE)$$

Where:

$\bar{x}$ = sample mean

z = z-score for your confidence level (95% CI $\rightarrow z = 1.96$)

SE = Standard Error = $SD / \sqrt{n}$

95% Confidence Interval: $\bar{x} \pm 1.96 \times (SD / \sqrt{n})$

99% Confidence Interval: $\bar{x} \pm 2.576 \times (SD / \sqrt{n})$

Python Example

```
import numpy as np
from scipy import stats as st

data = np.array([5, 7, 8, 9, 10]) # cleaned data (outlier removed)

mean = np.mean(data) # 7.8
se = st.sem(data) # Standard Error

# 95% confidence interval
ci_low, ci_high = st.t.interval(
    confidence=0.95,
    df=len(data)-1, # degrees of freedom
    loc=mean,
    scale=se
)
```

```
print(f'Mean : {mean:.2f}')
print(f'95% CI: ({ci_low:.2f}, {ci_high:.2f})')
# Output: 95% CI: (5.41, 10.19)

# Interpretation: we are 95% confident the TRUE mean
# of the population falls between 5.41 and 10.19.
```

Key Points to Remember

Concept	Explanation
Wider CI	Less precise — could be caused by small sample or high variance
Narrower CI	More precise — larger sample size or lower variance
95% confidence	Does NOT mean 95% of data falls in that range. It means the METHOD produces correct intervals 95% of the time.
Relationship to HT	If CI does NOT include the null value (e.g. 0), you can reject H_0

How Hypothesis Testing & Confidence Intervals Connect

These two concepts are two sides of the same coin:

Hypothesis Testing: Gives you a YES / NO answer (reject or not reject H_0)

Confidence Interval: Gives you a RANGE answer (where the true value likely lies)

Both use p-values, SE, and the normal distribution under the hood.

They are both tools for making DECISIONS FROM SAMPLE DATA.

In ML: These are used to evaluate models — is my model actually better than the baseline, or did it just get lucky with this sample?

Final Cheat Sheet

Measure / Concept	One-Line Summary
Range	Gap from smallest to largest — quick and dirty spread estimate
IQR	Spread of the middle 50% — ignores outliers — best for skewed data
Variance	Average of SQUARED distances from mean — in squared units
Standard Deviation	Square root of variance — back in original units — most common spread measure
Standard Error	$SD / \sqrt{n}$ — how accurate your sample mean estimate is
Symmetric Data	Both sides mirror each other — mean $\approx$ median
Right-Skewed	Tail on the RIGHT — Mean $>$ Median — a few very large values pull it right
Left-Skewed	Tail on the LEFT — Mean $<$ Median — a few very small values pull it left
Degrees of Freedom	Number of values free to vary — for samples: $n - 1$ — use <code>ddof=1</code> in code
Outlier	Any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$
Winsorize	Replace outliers with boundary values — keeps dataset size the same
Log Transform	Squashes large values — fixes right-skewed data
Hypothesis Testing	$p < 0.05 \rightarrow$ reject $H_0 \rightarrow$ statistically significant result
Confidence Interval	$\bar{x} \pm 1.96 \times SE$ — the range where the true mean likely lives (95% CI)